

AI Challenge - Stock Prediction Tool

In this project, our team created a tool that, when given historic stock data for a specific company, will predict future stock prices based on just a given day. This allows it to be very flexible as the model does not require much outside data.

We implemented our solution in python with a command line tool. The demo video is linked below:

<https://drive.google.com/file/d/1bP3MZHaFmZpSjRshWcuExBEUMCCmLCe9/view?usp=sharing>

Github repository:

<https://github.com/SquashyYoshi/AI-Challenge>

Project narrative:

<https://docs.google.com/document/d/1P21OBiidXwBhuYK5LIXt-4gYCtNR8befXG1MiEm5c/edit?usp=sharing>

Certification of Originality

We ensure that all code submitted was created for this AI challenge. In order to complete the challenge, we used the python library Sci-kit learn as well as [this dataset](#).

Primary project file (main.py):

```
from sklearn import metrics
from sklearn.ensemble import GradientBoostingRegressor
import csv
import sys
import numpy as np
from pathlib import Path
```

```
FILENAME = "main.py"
```

```
DATASET_PATH = "stock_market_data/forbes2000/csv/"
```

```
def main():
```

```
    result = parseArgs()
```

```
    if result[0] == "search":
```

```
        searchStocks(result[1])
```

```
    return
```

```

else:
    result = result[0]
    print("Preparing data...")
    dataList = csvToDict(result)
    if dataList[0] == "error":
        print(f"Invalid filename, use python {FILENAME} search {{stock symbol}} to find the correct company\n")
        return
    dates = []
    openPrice = []
    for row in dataList:
        #if that date or price is empty, it gets skipped
        try:
            open = float(row["Open"])
            date = row["Date"]
        except ValueError:
            continue
        openPrice.append(float(row["Open"]))
        dates.append(row["Date"])

    #formats input data into arrays of dates
    X = np.array(dateToNum(dates)).reshape(-1, 1) #sklearn expects a 2d array, we need
    to convert to 2d
    y = openPrice

    #splits it so that the first 80% of data is training data last 20% is testing data
    split = int(len(X) * 0.8)
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y[:split], y[split:]

    model = GradientBoostingRegressor(
        n_estimators=500,
        learning_rate=0.03,
        max_depth=5,
        min_samples_split=4,
        min_samples_leaf=2,
        random_state=42,
        subsample=0.8
    )

```

```

print("Training model...")
model.fit(X_train, y_train) #trains the model
y_pred = model.predict(X_test)
print("\nResults:")
print(f"Accuracy: {round(100 - (metrics.mean_absolute_percentage_error(y_test,
y_pred)*100.0), 4)}%")

```

```

print("\n=====")
print("Enter dates to predict stock prices (or 'quit' to exit)")
print("=====")
while True:
    userInput = input("\nEnter date (dd-mm-yyyy) or 'quit': ").strip()

    if userInput.lower() == 'quit':
        print("Exiting...")
        break

    try:
        #splits the date into 3 strings of day month year
        parts = userInput.split('-')
        if len(parts) != 3:
            print("Invalid format. Use dd-mm-yyyy")
            continue

        day, month, year = int(parts[0]), int(parts[1]), int(parts[2])

        #make sure the date is reasonable
        if not (1 <= day <= 31 and 1 <= month <= 12 and year > 1950):
            print("Invalid date values")
            continue

        #ensure format is dd-mm-yyyy
        userInput = f"{day:02d}-{month:02d}-{year}"

        #find dateNum relative to start date of dataset
        thisDateNum = dateToNum([dates[0], userInput])[1]
        thisDateNum = np.array(thisDateNum).reshape(1, -1)

```

```

prediction = model.predict(thisDateNum)[0]

actualPrice = None
actualDateUsed = None
#try to find exact date match
for i, date in enumerate(dates):
    if date == userInput:
        actualPrice = openPrice[i]
        actualDateUsed = date
        break
#if no exact match, find nearest date to entered date
if actualPrice is None:
    #check if date in range of dataset
    if X[0][0] <= thisDateNum <= X[-1][0]: #X[-1] is last date number
        #find nearest date num
        nearestIdx = 0
        nearestDistance = abs(X[0][0] - thisDateNum)
        for i in range(len(X)):
            distance = abs(X[i][0] - thisDateNum)
            if distance < nearestDistance:
                nearestDistance = distance
                nearestIdx = i

        actualPrice = openPrice[nearestIdx]
        actualDateUsed = dates[nearestIdx]

#2f rounds to 2 decimal places
print(f"\nPredicted price: ${prediction:.2f}")
if actualPrice is not None:
    print(f"Actual price: ${actualPrice:.2f}")
    if actualDateUsed != userInput:
        print(f"(Using nearest date in dataset: {actualDateUsed})")
    error = abs(prediction - actualPrice)
    print(f"Error: ${error:.2f}")
else:
    print("Actual price: Not available in dataset")

except (ValueError, IndexError) as e:
    print("Invalid input. Please use format dd-mm-yyyy")

```

```

#makes each str in dates be represented as number of days after startDate
def dateToNum(dates):
    #30.41 is average number of days in a month
    X = [] #0 represents first date in dataset, all other numbers represent amount of days
    after the start day
    X.append(0)
    startNum = int(round((getMonth(dates[0]) * 30.41) + getDay(dates[0]) +
(getYear(dates[0]) * 365)))
    largestDateNum = 0
    for date in dates:
        if date == dates[0]:
            continue
        thisDateNum = int(round((getMonth(date) * 30.41) + getDay(date) + (getYear(date)
* 365))) - startNum
        if thisDateNum < 0:
            print(f"****ERROR: Calculated negative date number {thisDateNum}, ignoring
data point**** ")
        else:
            X.append(thisDateNum)
            if thisDateNum > largestDateNum:
                largestDateNum = thisDateNum
    return X

def getMonth(date):
    return int(date[3:5])

def getDay(date):
    return int(date[0:2])

def getYear(date):
    return int(date[6:])

#searches through dataset for stock names matching the search
#prints a list of matches
def searchStocks(searchTerm):
    searchTerm = searchTerm.lower()
    csvDir = Path(DATASET_PATH)

    matches = []

```

```

for csvFile in csvDir.glob("*.csv"): #grabs all csv files from the folder
    if searchTerm in csvFile.stem.lower():
        matches.append(csvFile.name)

if matches:
    print(f"Found {len(matches)} matching stock(s):")
    for match in sorted(matches):
        print(f" {match}")
    print()
else:
    print(f"No stocks found matching '{searchTerm}'")

#reads through the csv file and converts it to a list of dictionaries
def csvToDict(filename):
    try:
        dataList = []
        with open(filename, 'r') as file:
            reader = csv.DictReader(file)
            for row in reader:
                dataList.append(row)
    except FileNotFoundError:
        return ["error"]
    return dataList

#goes through command line arguments and passes input into main
def parseArgs():
    try:
        if(sys.argv[1].lower() == "search"):
            return ["search", sys.argv[2]]
        else:
            #make sure filename ends in .csv
            filename = sys.argv[1].upper().removesuffix(".CSV")
            filename += ".csv"
            filename = DATASET_PATH + filename
            return [filename]
    except IndexError: #when index out of range, means no arguments were given
        print(f"Usage: python {FILENAME} {{stock symbol}}\n      python {FILENAME} search {{stock symbol}}")
        return ["error"]

```

main()